

Servermore: Serverless platform

Bc. Vojtech Babinský, Bc. Lukáš Častven, Bc. Matúš Koleják, Bc. Andrej Streicher

May 13, 2026

1 The goal

This project is about building a "serverless" (FaaS - Function as a Service) platform, inspired by ones like AWS Lambda. The goal is create a system, which can based on incoming HTTP requests dynamically start instances, process the request and then teardown the instance. Thus removing the need of a permanently running server.

2 Architecture

Architecture is split into multiple components, and is visualized in Figure 1:

1. **API Gateway:** Service, which accepts incoming requests, transforms them into a inner system representation and sends them further into the system.
2. **Commander:** The "brain" of the whole system. Decides which instance to use for incoming requests, whether to create new one (cold start), or reuse already running one (warm). Also responsible for storing user function binaries and serving them to the rest of the system.
3. **Runner:** Component running on execution nodes, which process the incoming requests by executing user's functions.
4. **Guest:** The user's submitted functions. They take in the request, process it, and return a response.

The whole request path is shown in Figure 2. A client sends an HTTP request to the Gateway, which strips the function identifier from the path and asks the Commander where to route it. If a healthy warm instance is already running, the Commander returns that runner and instance immediately. Otherwise it asks a Runner to prepare a new instance, download the binary if needed, start the Guest, and return the new instance identifier. The Gateway then forwards the invocation to the selected Runner, which calls the Guest over gRPC and returns the response back to the client.

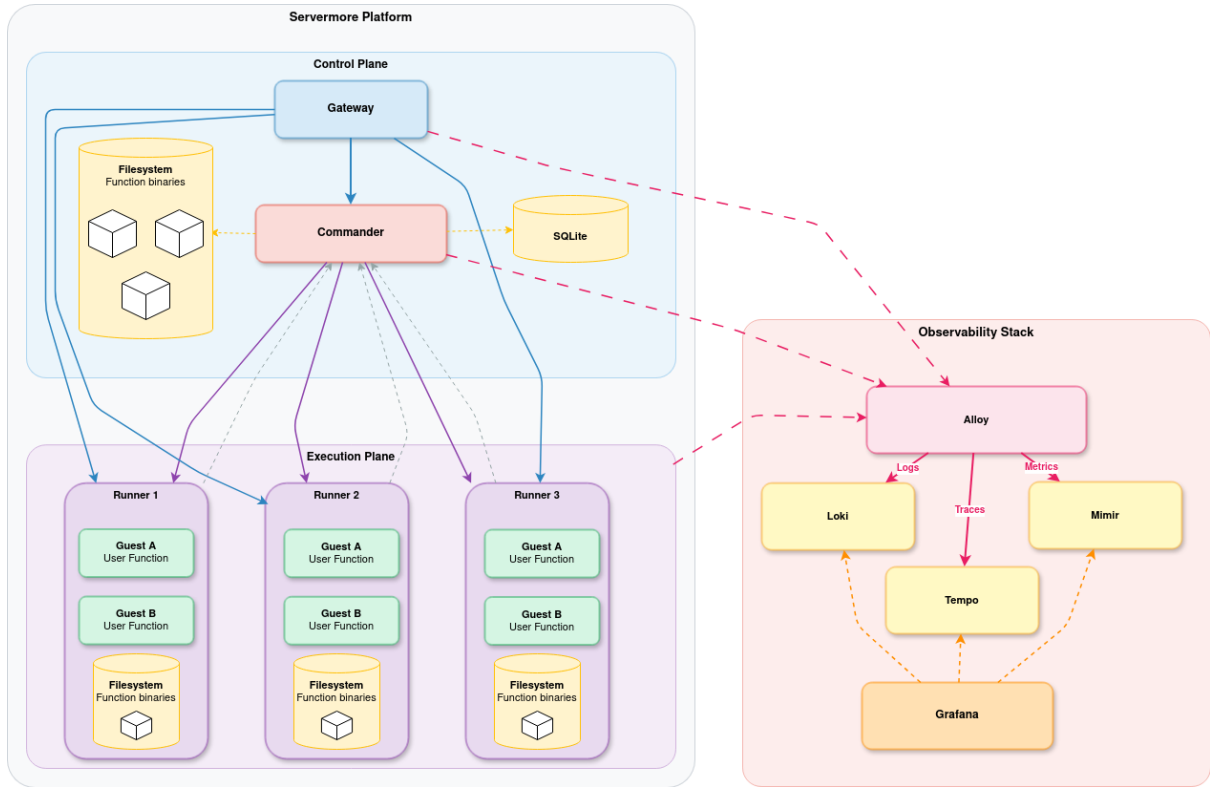


Figure 1: Architecture of the platform

3 Tech stack

The platform is built with the following technologies:

- **Golang:** used as the programming language go.dev.
- **HTTP:** used for public ingress.
 - The Gateway accepts client requests over HTTP.
 - The Commander function binaries API is also HTTP-based. It is defined with [OpenAPI](https://openapi.dev) and generated with [oapi-codegen](https://oapi-codegen.dev).
- **gRPC:** used for inner system communication grpc.io.
- **SQLite:** used by Commander for storing data sqlite.org. We chose it because we did not need to complicate the stack more and SQLite is enough for now; because the SQLite implementation is behind an interface, we could swap it for PostgreSQL easily.
 - Query code is generated with [sqlc](https://github.com/mattn/go-sqlc) to keep database access type-safe.
- **Structured logging:** implemented with Go's [slog](https://pkg.go.dev/log).
- **Monitoring:**
 - **Instrumentation:** based on [OTEL](https://opentelemetry.io).
 - * HTTP server instrumentation uses [otelchi](https://otelchi.dev).
 - * gRPC instrumentation uses [otelgrpc](https://otelgrpc.dev).
 - **Visualization:** provided by the [Grafana](https://grafana.com) stack.

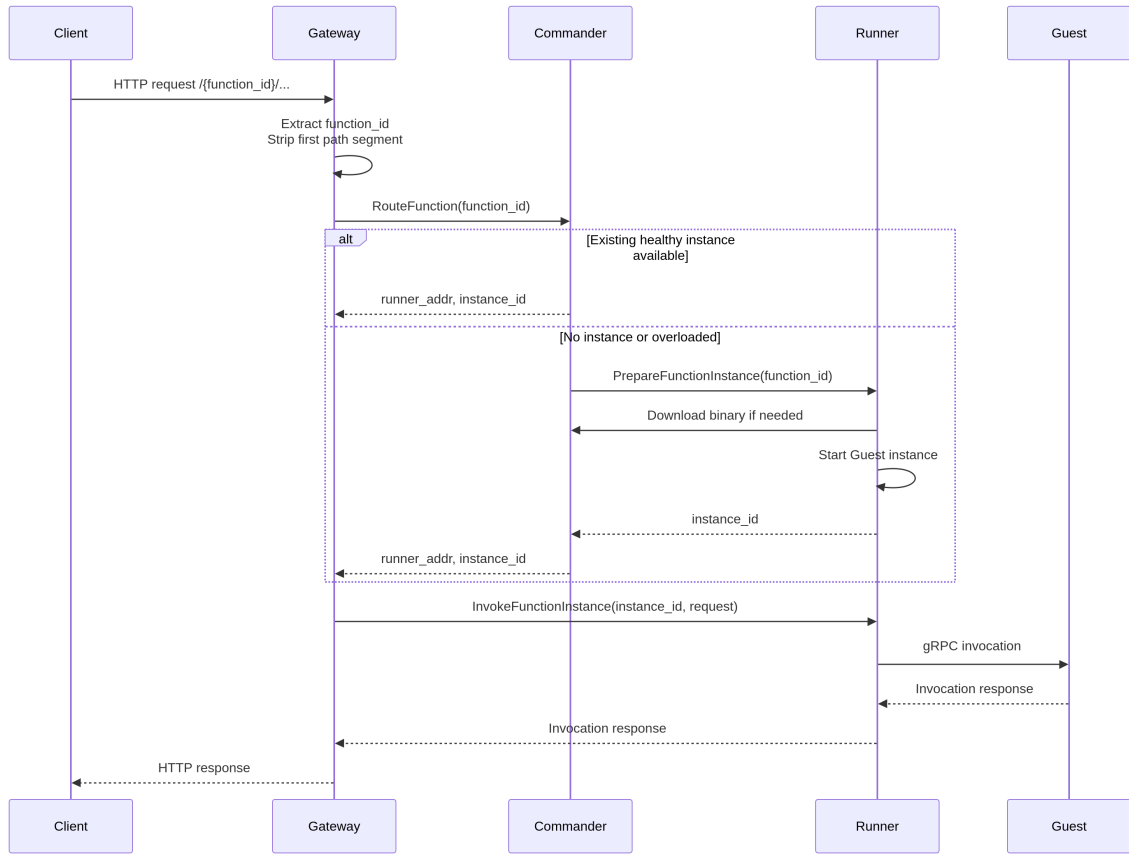


Figure 2: High-level request flow through the platform

4 API Gateway

The API Gateway is the stateless public HTTP endpoint of the system. It receives requests from clients, extracts the function identifier from the URL, normalizes the request into the internal format used by the platform, and then through gRPC asks the Commander to which runner the request should be routed to. After Commander returns the selected runner and instance identifier, the Gateway forwards the invocation to that runner's gRPC API and sends the response back to the client. Simplified sequence flow of this can be seen in Figure 2.

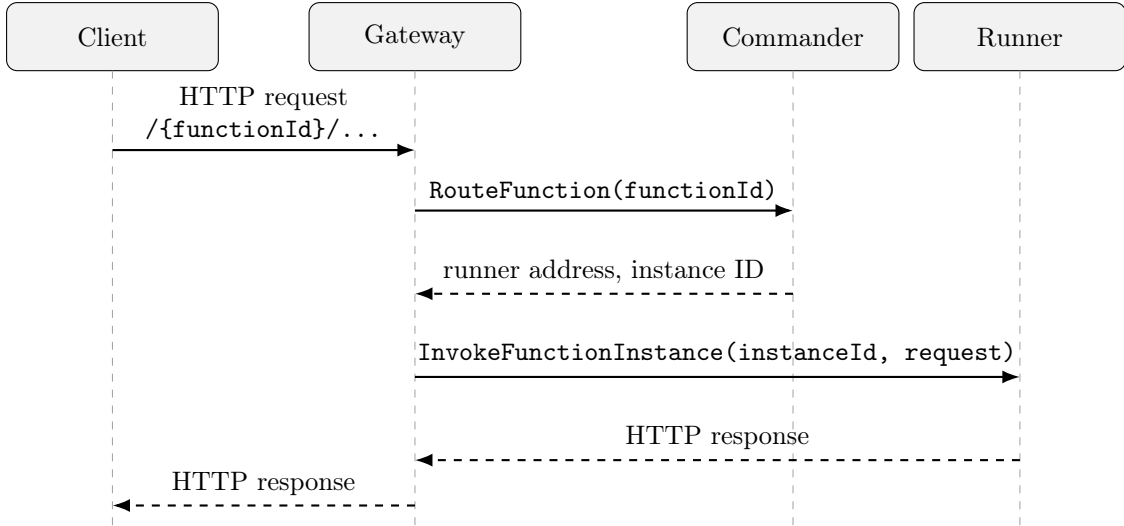


Figure 3: High-level request flow through the API Gateway

5 Commander

The Commander is the control plane of the platform. It keeps track of the current system state and decides where each request should be executed. Conceptually, it acts as the brain of the system: it knows which runners are available, which function instances are already running, and whether a request should reuse an existing warm instance or trigger a cold start.

To maintain this knowledge, the Commander periodically polls runners for heartbeats. Those heartbeats report runner availability, queue depths, and resource usage, so the Commander can avoid overloaded or unavailable runners when making routing decisions.

The Commander also exposes an HTTP API for uploading and downloading function binaries. Uploaded functions are stored and later served to runners when a new instance needs to be started and the runner doesn't have the function copy locally. More in subsection 5.1.

Commander exposes a gRPC API used for registering new runners and for evaluating where to route requests. The definition file is in `pkg/commander/grpc/commander.proto`.

```

service Commander {
  rpc Heartbeat(HeartbeatRequest) returns (HeartbeatResponse) {}

  rpc RegisterRunner(RegisterRunnerRequest) returns (RegisterRunnerResponse) {}

  rpc RouteFunction(RouteFunctionRequest) returns (RouteFunctionResponse) {}
}
  
```

The implemented routing logic is depicted in Figure 4.

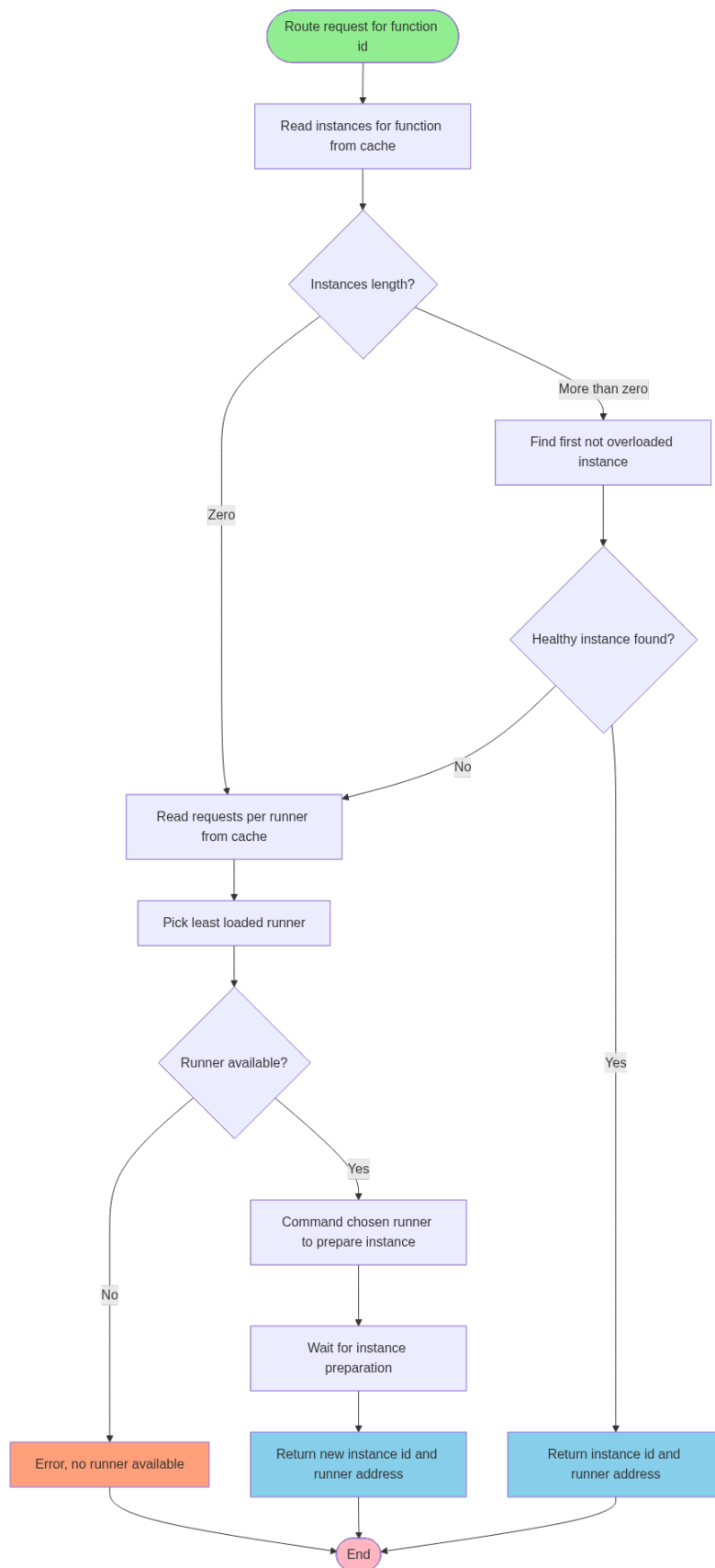


Figure 4: Routing logic flow

Commander persists its state in SQLite, which stores the function catalog and the registered runners, the database schema is very simple:

```
create table functions (  
    id integer primary key,  
    hash blob not null unique,  
    name text not null,  
    path text not null,  
  
    created_at datetime not null default current_timestamp,  
    updated_at datetime not null default current_timestamp  
);  
  
create table runners (  
    id integer primary key,  
    addr text not null unique,  
  
    created_at datetime not null default current_timestamp,  
    updated_at datetime not null default current_timestamp  
);
```

At runtime, Commander also keeps an in-memory routing cache with the data used to answer routing decisions quickly. The implementation only now supports a in-memory one, but can easily be replaced by some other cache like Valkey. The new implementation would just need to adhere to this interface:

```
type RoutingCache interface {  
    // FunctionIdInstances returns a map of instance ids to their (last known)  
    // request queue length  
    FunctionIdInstances(ctx context.Context, funcId string) (map[string]int, error)  
  
    // RunnerAddressOfInstance finds the runner address on which the instance  
    // is running  
    RunnerAddressOfInstance(ctx context.Context, instanceId string) (string, error)  
  
    // RequestsPerRunner returns a map of runner address to the number of  
    // (last known) request count in all of it's queues.  
    RequestsPerRunner(ctx context.Context) (map[string]int, error)  
  
    // StatsPerRunner returns (last known) runner resource metrics  
    StatsPerRunner(ctx context.Context) (map[string]ResourceMetrics, error)  
  
    // CachedInstancesCount returns the number of cached instances.  
    CachedInstancesCount(ctx context.Context) (int, error)  
  
    // SetInstance registers or updates a function instance and its runner.  
    SetInstance(ctx context.Context, funcId, instanceId, runnerAddr string, queueLen int) error  
  
    // UpsertRunnerHeartbeat replaces the cached heartbeat data for a runner.  
    UpsertRunnerHeartbeat(  
        ctx context.Context,  
        runnerAddr string,  
        queueDepths map[string]uint32,  
        metrics ResourceMetrics,  
    ) error
```

5.1 Functions management

The Commander also exposes an HTTP API for function management. It is used to deploy a function binary: the user uploads the binary to Commander, Commander saves it on file system storage, persists the function name, function ID, and storage path to the database, and then returns the function ID back to the user (Figure 5)

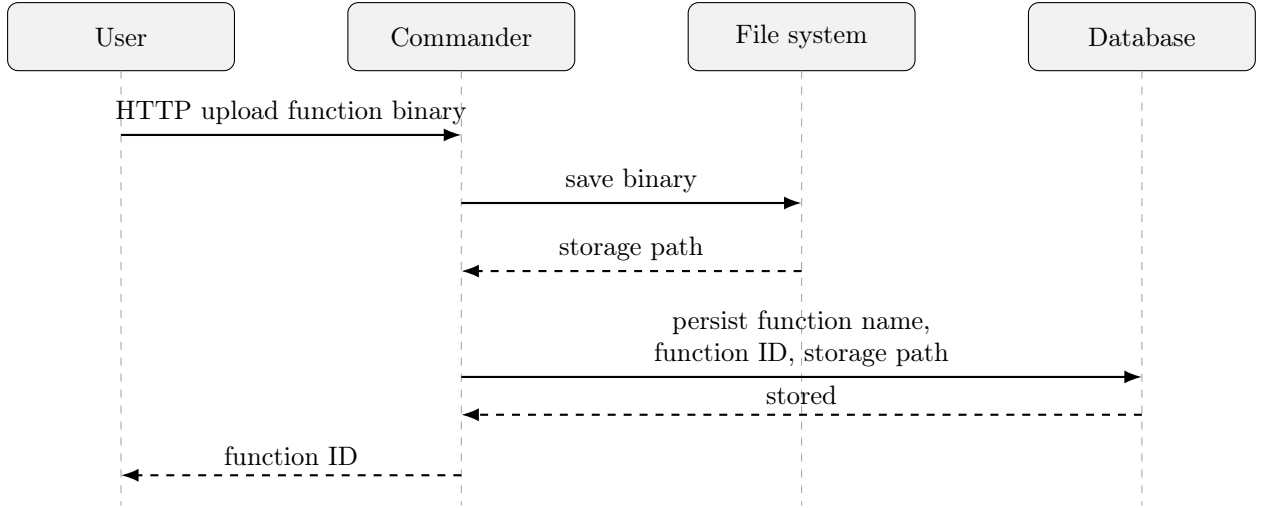


Figure 5: Function deployment flow in Commander

6 Runner

The Runner is the most complex part of the platform. It has to coordinate parallel instance startup, shutdown, binary downloads, and request execution at the same time, while still merging concurrent requests for the same function into one starting instance instead of spawning duplicate processes.

This component exposes a gRPC API which provides heartbeat data needed for routing logic, procedure for preparing instance of a function, and then procedure for invoking a request inside an instance. The definition file is in `pkg/runner/grpc/runner.proto`.

```

service Runner {
  rpc Heartbeat(HeartbeatRequest) returns (HeartbeatResponse) {}

  rpc PrepareFunctionInstance(PrepareInstanceRequest) returns (PrepareInstanceResponse) {}

  rpc InvokeFunctionInstance(InvokeInstanceRequest) returns (InvokeInstanceResponse) {}
}
  
```

The coordination model is the main challenge. The runner keeps track of instances that are already running, instances that are still starting, and downloads that are already in flight. When several requests arrive at once, they all wait on the same startup or download result, which lets them collapse into one shared preparation path. This can be better seen in the Figure 6, in which first (the bottom 2 ones) requests take 1.1 second because they are waiting for the instance to start, and then subsequent requests are executed in 2-3 milliseconds.

Table - Traces				
Trace ID	Start time	Service	Name	Duration
> 31f0534fc72359ad4b...	2026-05-12 14:52:07	gateway	/functionid/	2 ms
> a3920a2d74bec90e2fb...	2026-05-12 14:52:05	gateway	/functionid/	3 ms
> 5aa3260807fa969bee5...	2026-05-12 14:52:05	gateway	/functionid/	2 ms
> d0d8f3a5badb1c3fe54...	2026-05-12 14:52:04	gateway	/functionid/	2 ms
> f7445d58883c065341...	2026-05-12 14:52:02	gateway	/functionid/	2 ms
> 55a935d30a9d6423b5...	2026-05-12 14:52:02	gateway	/functionid/	2 ms
> ea8de401d3875c5482...	2026-05-12 14:52:01	gateway	/functionid/	3 ms
> 18942bbd7ed9eefcdf6...	2026-05-12 14:51:59	gateway	/functionid/	603 ms
> 62adcc5190b67937ca4...	2026-05-12 14:51:59	gateway	/functionid/	1.10 s
> e0c70d34fc785609687...	2026-05-12 14:51:58	gateway	/functionid/	1.10 s

Figure 6: Warm instance reuse keeps subsequent requests in the millisecond range

Once an instance is running, requests are put and processed through its queue, and idle instances are stopped after the configured timeout. Figure 7 shows how count of active instances of the same function is increased throughout all the runners (in the test setup there were 3 runners, they are represented by the different colors in the graph) and then when there is no traffic, the instances are scaled down to zero.

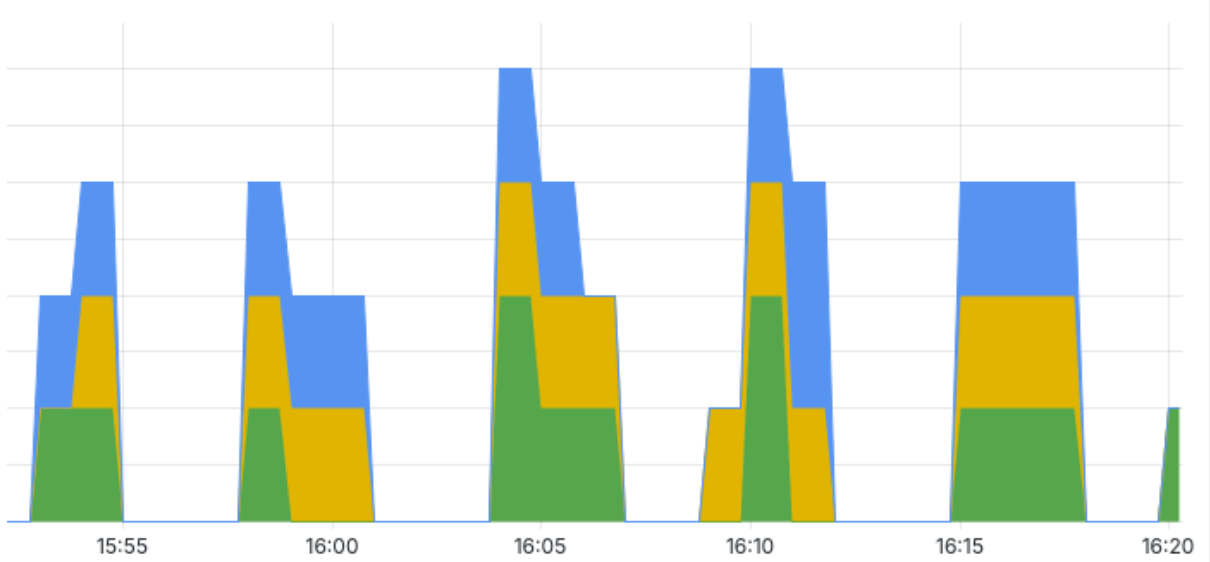


Figure 7: Active instances increase with load and drop to zero when idle

6.1 Instance runtime

When runner is asked to prepare an instance for a function, it does:

1. Checks if a function binary is stored in its local storage.
 - (a) If not, downloads it from the Commander's HTTP API.
2. Launches the binary in a runtime.
3. Calls the guest's gRPC heartbeat procedure (more about this in Section 7) until a successful response is returned. This shows how the runner polls the starting instance until it is up and ready to process requests.
4. Returns the id of this newly started one.

Right now only the native runtime is implemented, where the binary is started directly on the host, a free port is assigned, and the guest is expected to serve gRPC on that port. The guest API is on gRPC network layer, so adding another runtime would not be difficult: a Docker runtime could copy the binary into a container and expose the gRPC port, or a JavaScript runtime could do the same with raw JS source and a container that provides a JS engine.

7 Guest

Guests are the user functions themselves. Runner monitors and passes requests through the guest's gRPC API. The definition file of this server in `pkg/guest/guest.proto`.

```

service Guest {
  rpc Heartbeat(HeartbeatRequest) returns (HeartbeatResponse) {}

  rpc InvokeFunction(InvocationRequest) returns (InvocationResponse) {}
}

```

At the moment we provide only a Go SDK, which wraps the function and starts the gRPC server expected by the runtime. For other languages, SDKs for them would be similar, because gRPC implementations exists for many languages.

The only requirement from the runtime is that it provides the environment variables needed to configure the guest server, such as the address it should bind to.

7.1 Constraints

Since a guest communicates with the runner over gRPC, it has to contain a gRPC server. This means that user's functions must be written in a language that supports gRPC with suitable libraries (they can cause the overall binary to be bigger).

In order to support other languages, servermore would need to provide SDKs for those languages, similar to the Go SDK, which is currently used.

8 Monitoring

The monitoring stack is built around the Grafana stack. The commander, gateway, and runners export OTLP data to Alloy over gRPC. Alloy reads the Docker container logs of those components. Then forwards logs to Loki, traces to Tempo, and metrics to Mimir. Grafana is provisioned with all three datasources, including trace-to-log correlation from Tempo into Loki. Figure 8 displays the monitoring setup.

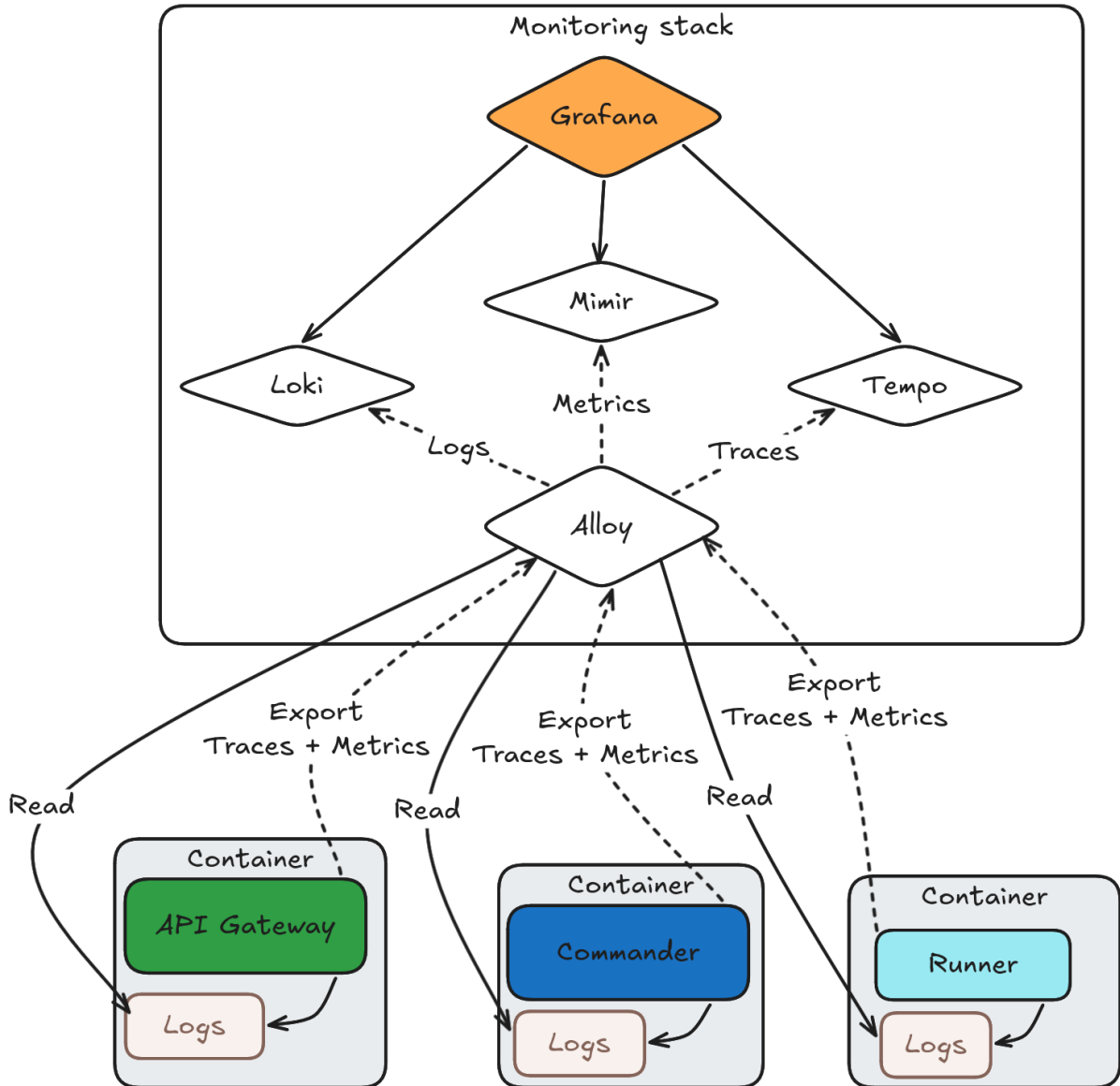


Figure 8: Monitoring stack and telemetry flow

8.1 Instrumentation setup

Tracing is implemented as middleware around the HTTP and gRPC stacks. The trace starts before the request logic runs and finishes after it returns, so the full request path is captured around the business code.

Metrics are collected inside the request processing logic itself. That makes them describe the actual work done during the request instead of only wrapping the handler boundary.

The instrumentation uses `otelchi` for HTTP middleware, `otelgrpc` for gRPC middleware, and the OpenTelemetry Go SDK packages for traces, metrics, exporters, and resource setup.

8.2 Example

An example of tracing a request is visualized in the following Figure 9. It shows a single request flowing through the Gateway, Commander, Runner, and Guest. Because the same trace context is propagated through the HTTP and gRPC hops, the full request stays in one trace and the

logs can be linked back to the same span tree.

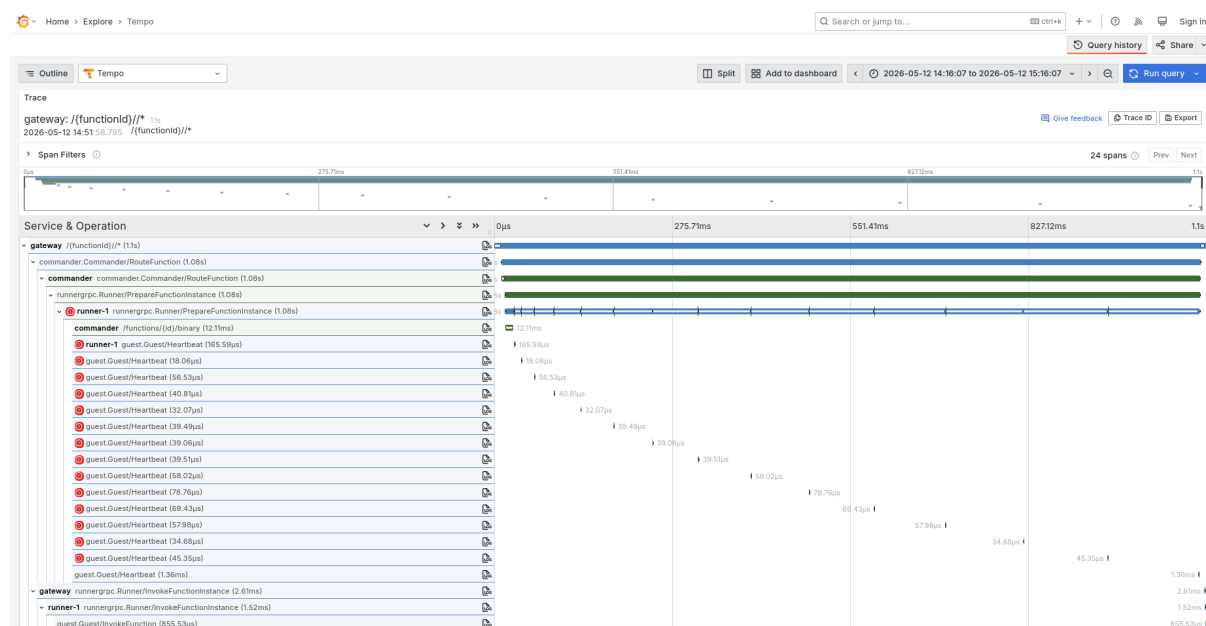


Figure 9: Grafana trace view for a request passing through the platform

9 Setup

To run Servermore locally, Docker must already be installed and usable without `sudo`, including the Docker Compose. After that, install `mise` and run `mise install`. That installs the project toolchain and generators declared in `mise.toml`, including Go, `sqlc`, `oapi-codegen`, and `protoc`.

9.1 Scalability testing scenario

Running `mise local-test` opens the tester TUI. It first compiles the example functions and then starts `docker/docker-compose.local-test.yaml`. The first startup takes a while because Docker has to pull the Grafana stack images and build the Servermore service images. Once the stack is ready, the TUI can deploy a function, and after deployment it starts sending requests to that function automatically. When the TUI quits, it brings the stack down and removes the Compose volumes.

Figure 10 shows how the Docker Compose stack is deployed and connected for the scalability test.

The TUI request settings are:

- **Batch size:** how many requests the TUI sends in one burst before it pauses.
- **Requests/s:** the request rate inside a burst.
- **Delay (s):** how long the TUI waits between bursts.

Use `0` to enable or disable request sending, `up/down` or `k/j` to select a setting, `+/` to change it, and `enter` to deploy the selected function. The TUI also shows a Grafana link and can start Dozzle on demand for quick container log browsing (dozzle.dev).

The purpose of the test is to deploy a function and generate traffic so the scaling behavior is visible.

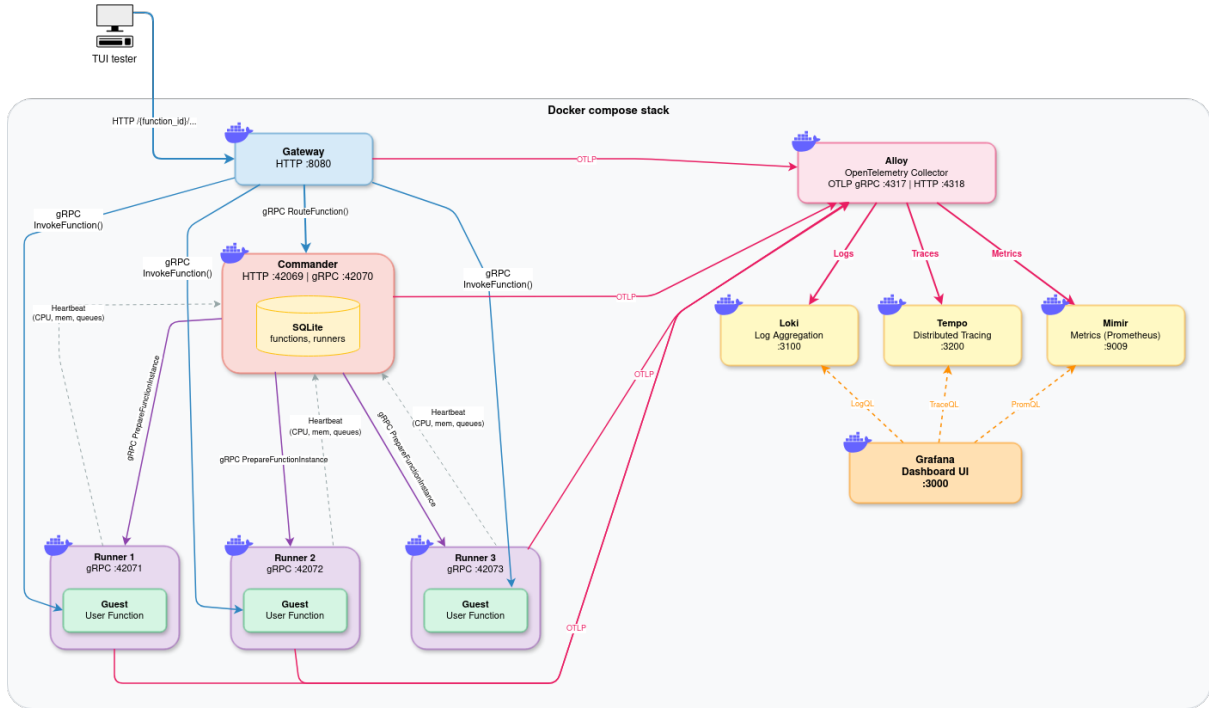


Figure 10: Docker Compose stack used for the scalability test

9.1.1 Grafana

To explore the test yourself open Grafana and use **Explore**. Loki shows logs, Tempo shows traces, and Mimir provides metrics through the Prometheus datasource. The trace view shows the full request path through the gateway, commander, runner, and guest. Logs and traces are not real-time, expect roughly a minute before they appear in Grafana.

10 Conclusion

Servermore is a working little version of a serverless function platform. The current version handles cold starts and warm reuse, routes requests through the Commander, and executes user functions through the Runner and Guest layers.

The project also shows the value of observability. With traces, logs, and metrics in Grafana, it is possible to follow a request across HTTP and gRPC boundaries and understand how the platform behaves under load. The local test setup makes this easy to reproduce and inspect end to end.

Future work could focus on better scheduling, stronger isolation for guest execution, persistent state outside SQLite, and support for more runtimes and languages.

Work split

Author	Work split
Andrej	Commander cache, Runner monitoring
Lukas	Architect, Runner, Guest SDK, docs
Matüs	Routing, tests, docs
Vojto	Gateway, monitoring